Map<K,V> interface :

Map<K,V> is a predefined interface available in java.util package from JDK 1.2V

As we know Collection interface is used to hold single OR individual object but Map interface will hold group of objects in the form key and value pair. {key = value}

Map<K,V> interface is not the part the Collection, It is a separate interface.

Before Map interface We have Dictionary<K,V>(abstract class) class and it is extended by Hashtable<K,V> class in JDK 1.0V

Map interface works with key and value pair.

Here key and value both are objects.

Here key must be **unique** and value may be **duplicate**.

key (username)	**value (password)**
ravi@gmail.com	ravi1234
scott@gmail.com	scot^&&*
raj@gmail.com	raj)*^#%%^

entry

In this diagram, we have 3 entries are available

Each key and value pair is creating one Entry.(Entry is nothing but the combination of key and value pair)

```
public interface Map<K,V>
{
    interface Entry<K,V>
    {
      //key and value
    }
}
```

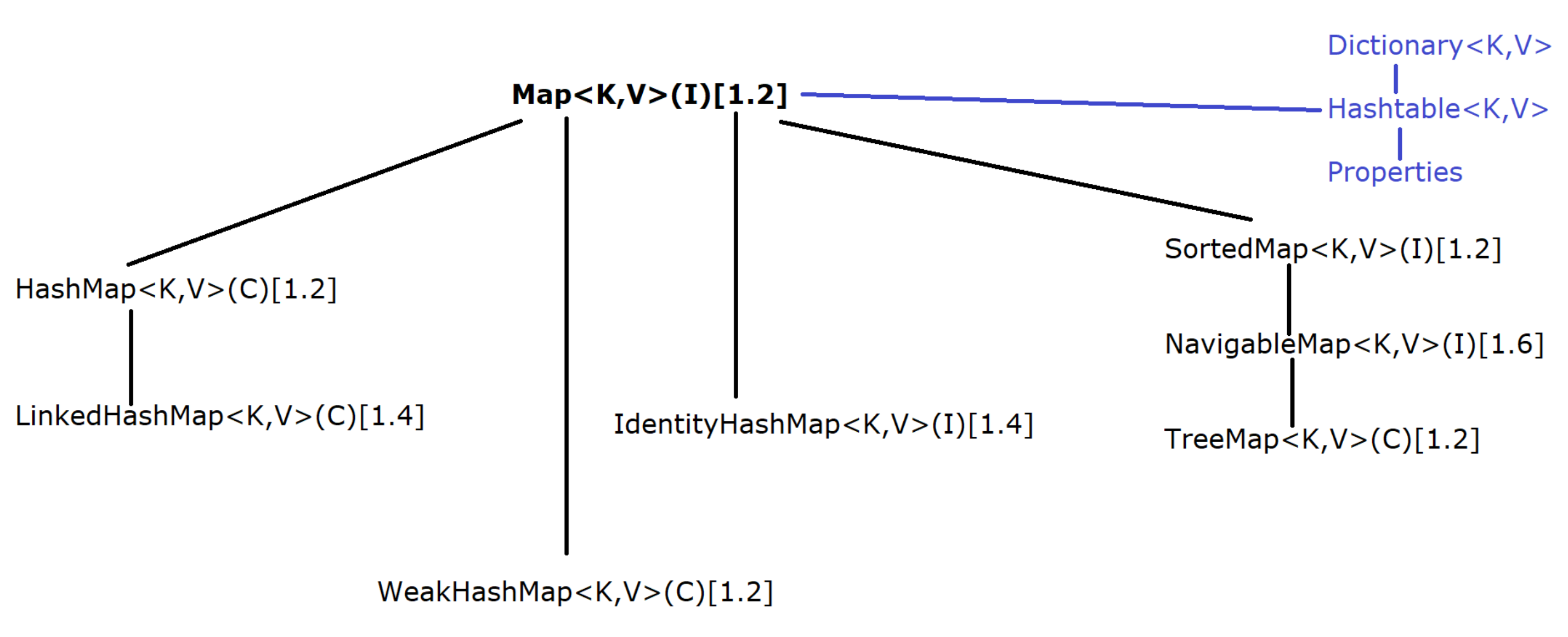
How to represent this entry interface (Map.Entry in .java) [Map$Entry in .class]

In Map interface whenever we have a duplicate key then the **old key value** will be replaced by **new key(duplicate key) value.**

Map interface has defined forEach(BiConsumer<T,U> cons) method to work with group of Objects.It does not extends Iterable interface.

Iterator and Spliterator we can't work **directly** using Map.

Map interface Hierarchy :



Methods of Map<K,V> interface :
---

1) V put(K key, V value) :- To insert one entry in the Map collection. It will return the value of old Object key, if the key is already available(Duplicate key), If key is not available (Unique key) then it will return null.

2) default V putIfAbsent(Object key, Object value) :- It will insert an entry, if and only if, key is not available , if the key is already available then it will not insert the Entry to the Map Collection.

3) V get(Object key) :- It will return corresponding key value, if the key is not available then it will return null.

4) default V getOrDefault(Object key, Object defaultValue) :- To avoid null value this method has been introduced as an alternate of get() method from JDK 1.8V, here we can pass some defaultValue to avoid the null value.

5) boolean containsKey(Object key) :- To Search a particular key

6) boolean containsValue(Object value) :- To Search a particular value. (value may be duplicate so it will search the first occurrance)

7) int size() :- To count the number of Entries.

8) V remove(Object key) :- One complete entry will be removed based on the key only. WIll return null if key is not available OR removed key value, if key is available

9) public boolean remove(Object key, Object value) :- It will only delete the entry, if the key and value both are matching.

10) void clear() :- Used to clear all the entries of the Map

11) boolean isEmpty() :- To verify Map is empty or not?

12) void putAll(Map m) :- Merging of two Map collection

13) default void forEach(BiConsumer<T,U> cons) :- Used to fetch key and value from the Map. It is added inside Map<K,V> interface from
JDK 1.8V

14) V computeIfAbsent(K key, Function<K,V> mapper) : It computes a value for a given key if it is not already present in the map, and then inserts that computed value into the map.

```
Map<String, String> map = new HashMap<>();
  map.computeIfAbsent("language", key -> "Java");
  map.computeIfAbsent("language", key -> "Python");
  System.out.println(map);
```

15) V computeIfPresent(K key, BiFunction<K,V,V> mapper) : It updates the value for a key, only if that key is already available in the map.

```
Map<String, Integer> map = new HashMap<>();
map.put("apple", 10);
map.put("banana", 5);
map.computeIfPresent("apple", (key, val) -> val + 5);
map.computeIfPresent("orange", (key, val) -> val + 5);
System.out.println(map);
```

Methods of Map interface to convert the Map into Collection :

We have Map interafce methods through which we can convert Map interface into Collection interface which is known as collection views method.

1) public Set<Object> **keySet()** : It will retrieve all the keys.

2) public Collection **values()** : It will retrieve all the values.

3) public Set<Map.Entry<K,V>> **entrySet()** : It will retrieve key and value both in a single object.

    a) getKey() : Will provide the key
    b) getValue() : Will provide the value

HashMap<K,V> [UNORDERED, UNSORTED, NO DUPLICATE KEY]
--

public class HashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>, Serializable, Clonable

It is a predefined class available in java.util package under Map interface available from JDK 1.2v.

It gives us unsorted and Unordered map. when we need a map and we don't care about the order while iterating the elements through it then we should use HashMap.

It inserts the element based on the hashCode of the Object key using hashing technique [hasing alogorithm]

It does not accept duplicate keys but value may be duplicate.

It accepts only one null key(because duplicate keys are not allowed) but multiple null values are allowed.

HashMap is not synchronized.

Time complexity of search, insert and delete will be O(1)

We should use HashMap to perform fast searching opeartion.

For eliminating duplicate keys in hashMap object we should compulsory follow the contract between hashcode and equals(Object obj) OR Use record

Iterator is Fail Fast Iterator.

It contains 4 types of constructors

1) HashMap hm1 = new HashMap();
   It will create the HashMap Object with default capacity is 16. The default load fator or Fill Ratio is 0.75
(75% of HashMap is filled up then new HashMap Object will be created having double capacity with re-hashing)

2) HashMap hm2 = new HashMap(int initialCapacity);
   will create the HashMap object with user specified capacity

3) HashMap hm3 = new HashMap(int initialCapacity, float loadFactor);
   we can specify our own initialCapacity and loadFactor(by default load factor is 0.75)

4)HashMap hm4 = new HashMap(Map<K,V> map);
   Interconversion of Map Collection.